

Ejercicios depuración

Cada ejercicio tiene 5 errores. Debes de localizar los errores mediante la depuración en los siguientes ejercicios:

1. Seguros Coche. Se quiere calcular el precio de un seguro de coche según:

Parámetro	Valor / Regla
Tipo de coche	“Turismo”, “Deportivo”, “Furgoneta”
Antigüedad (años)	0-2 → 0 % de recargo, 3-5 → 5 %, ≥6 → 15 %
Cobertura	“Básica” (sin recargo), “Completa” (+10 %)
Descuento por buen conductor	Si el conductor tiene menos de 3 multas → 5 % de descuento

El precio base del seguro es **500 €**.

Casos de prueba (salida esperada)

Caso	Entrada	Salida esperada
1	Tipo: Turismo Antigüedad: 1 Cobertura: Básica Multas: 0	Precio = 500 € (ningún recargo, 5 % de descuento → 475 €)
2	Tipo: Deportivo Antigüedad: 4 Cobertura: Completa Multas: 2	Recargo antigüedad 5 % → 525 € Recargo cobertura 10 % → 577,5 € Descuento 5 % → 548,63 €
3	Tipo: Furgoneta Antigüedad: 7 Cobertura: Básica Multas: 5	Recargo antigüedad 15 % → 575 € Sin recargo cobertura → 575 € Sin descuento → 575,00 €

2. Gestor Pedidos. Una tienda online gestiona pedidos. Cada pedido tiene:

Campo	Tipo
id	entero (autogenerado)
producto	cadena
cantidad	entero (>0)
precio_unitario	float
estado	"pendiente", "enviado" o "cancelado"

Se desea:

- Crear un pedido.
- Cambiar su estado.
- Calcular el importe total ($\text{cantidad} * \text{precio_unitario}$).

Casos de prueba (salida esperada)

Caso	Operación	Salida esperada
1	Crear pedido “Camisa”, 3 unidades, 20 €	id = 1, total = 60 €, estado = pendiente
2	Cambiar estado a “enviado”	estado = enviado (minúsculas)
3	total_ventas() tras crear dos pedidos (2 × 20 € + 1 × 80 €)	120,00 €
4	Intentar crear pedido con cantidad = 0	Debe lanzar ValueError (no permitido)

Caso	Operación	Salida esperada
5	<code>cambiar_estado("Cancelado")</code> (con mayúscula inicial)	Estado debe quedar cancelado (todo minúsculas)

3. Biblioteca. Se modela una biblioteca sencilla. Cada libro tiene:

Campo	Tipo
<code>isbn</code>	cadena (formato "XXX-X-XX-XXXXX-X")
<code>titulo</code>	cadena
<code>autor</code>	cadena
<code>copias</code>	entero (número de ejemplares disponibles)

Se necesita:

- Registrar libros.
- Prestar un libro (disminuir `copias`).
- Devolver un libro (aumentar `copias`).
- Consultar disponibilidad.

Casos de prueba (salida esperada)

Paso	Acción	Resultado esperado
1	Registrar libro con 3 copias	<code>catalogo</code> contiene el ISBN, <code>copias</code> = 3
2	Prestar 3 veces	Después de la tercera, <code>copias</code> = 0
3	Intentar prestar de nuevo	Levanta <code>RuntimeError</code> con mensaje "No hay copias disponibles"
4	Devolver una copia	<code>copias</code> pasa a 1 (no a 2)
5	Registrar otro libro con mismo ISBN	No debe sobrescribir (debe lanzar excepción o ignorar)

4. Gestor de Eventos

Una pequeña asociación organiza eventos (charlas, talleres, meet-ups). Cada evento tiene:

Campo	Tipo
<code>codigo</code>	cadena (identificador único, ej. EV001)
<code>nombre</code>	cadena
<code>fecha</code>	cadena en formato YYYY-MM-DD
<code>asistentes_max</code>	entero (capacidad máxima)
<code>inscritos</code>	entero (número de personas inscritas)
<code>estado</code>	"programado" , "completado" o "cancelado"

Se requiere una aplicación que permita:

1. **Crear** un evento.
2. **Inscribir** a una persona (incrementar `inscritos`).
3. **Cancelar** un evento.
4. **Marcar** un evento como completado (solo si la fecha ya pasó).

5. Consultar cuántos cupos quedan.

Casos de prueba (salida esperada)

Paso	Acción	Resultado esperado
1	Crear EV001 (30 plazas) y EV002 (20 plazas)	Ambos aparecen en el catálogo con estado = programado, inscritos = 0.
2	Inscribir 5 personas en EV001 y 3 en EV002	EV001.inscritos = 5, EV002.inscritos = 3.
3	Cancelar EV002	EV002.estado = cancelado.
4	Completar EV001 con fecha actual posterior (2024-06-16)	EV001.estado = completado.
5	Consultar total_cupos_libres	42 (30-5 + 20-3).
6	Intentar inscribir una sexta persona en EV001 (capacidad 30)	La inscripción se permite (inscritos = 6).
7	Intentar inscribir una vigésima primera persona en EV002 (capacidad 20)	Después de corregir el error 3, se lanza ValueError o se impide la inscripción.
8	Intentar crear otro evento con código EV001	Después de corregir el error 4, se lanza excepción indicando código duplicado.
9	Llamar a obtener_evento("XYZ") con código inexistente	Después de corregir el error 5, se lanza KeyError.
10	Llamar a completar_evento con fecha anterior al evento	Lanza ValueError (lógica de fechas ya es correcta).